
SnapStack

AI Receipt Scanner & Expense Tracker

DAY 04 | JULY 2, 2026

Snap a receipt. The AI does the bookkeeping.

Category	Details
Startup Name	SnapStack
Type	Offline AI Expense Tracker (Mobile-Only)
Stack	Vanilla HTML / JS + Ollama Vision Models
Models Supported	llava, llama3.2-vision, bakllava
Privacy	100% Local — Receipt Images Never Leave Device
Author	Mayank Swaraj / Mayank Creations
Series	Daily Local LLM Startup — Day 04

WHITEPAPER

SnapStack: Vision-AI Expense Tracking Without the Cloud

Abstract

SnapStack is a mobile-first, offline expense tracker that uses a local vision-capable LLM (via Ollama) to extract merchant name, amount, and category directly from a photographed receipt. No data — image or transaction — ever leaves the device. This paper covers the problem space, mobile-only product design, technical architecture, and market positioning.

1. Problem Statement

Personal finance apps overwhelmingly require bank account linking (Plaid, Mint, YNAB) or cloud OCR services for receipt scanning (Expensify). Both approaches expose sensitive financial data to third parties. Manual expense entry, the privacy-safe alternative, is tedious enough that most people abandon it within weeks. There has been no consumer-grade tool that combines the convenience of photo-based capture with complete on-device privacy — until local vision-language models became viable on consumer hardware in 2025-2026.

2. Solution

- Snap-to-extract — photograph a receipt, local vision model extracts merchant, amount, and category automatically
- Mobile-only design — single-column, thumb-reachable, camera-first interface built for phone use exclusively
- Editable AI extraction — auto-filled fields remain fully editable before saving, correcting any model error
- AI spending insights — on-demand pattern analysis across recent transactions
- Category-based visual organization — color-coded icons for instant scanning of expense history

3. Technical Architecture

Layer	Technology	Role
UI	Vanilla HTML5 / CSS3 (mobile-only)	Single-column layout, max-width 480px, capture-first flow
Logic	Vanilla JavaScript (ES6+)	Image capture handling, base64 encoding, expense state man
Storage	localStorage (browser)	Expense records persisted as JSON array
AI Runtime	Ollama (local process)	Vision-LLM inference for receipt extraction + text-LLM for insig
Vision Models	llava / llama3.2-vision / bakllava	Multimodal extraction of merchant, amount, category
Camera Input	HTML5 file input with capture="environment" / native mobile camera trigger, no custom camera UI needed	

4. Mobile-Only Design Rationale

SnapStack is deliberately built mobile-only — no responsive desktop layout. Receipt photography is an inherently mobile action: nobody scans a paper receipt with a desktop webcam. The interface uses a fixed max-width container, bottom-anchored capture bar (thumb zone), and bottom-sheet modals — all conventions borrowed from native mobile app design rather than responsive web design.

5. Market Opportunity

The personal finance app market was valued at approximately USD 1.6 billion in 2024, with receipt scanning and expense categorization representing core engagement features. Growing distrust of bank-linked finance apps following high-profile data breaches creates demand for a local-only alternative. SnapStack targets privacy-conscious freelancers, gig workers tracking deductible expenses, and budget-conscious consumers who want automation without surveillance.

6. Monetization Strategy

Tier	Price	Offering
Free	USD 0	Unlimited manual entry, 10 AI scans/month, 1 insight/week
Pro	USD 5/mo	Unlimited AI scans, unlimited insights, CSV export
Lifetime	USD 34	Everything in Pro, forever
Tax Pack	USD 19 one-time	Category-to-tax-deduction mapping, annual export

7. Competitive Advantage

Feature	SnapStack	Expensify	Mint	YNAB
Offline / Local AI	YES	NO	NO	NO
No Bank Linking Required	YES	Partial	NO	NO
Photo Receipt AI Extraction	YES	YES (cloud)	NO	NO
Zero Data Transmission	YES	NO	NO	NO
Single HTML File	YES	NO	NO	NO

02

SOURCE CODE PAPER

Annotated technical breakdown of SnapStack.html

SOURCE CODE PAPER

SnapStack.html — Annotated Technical Breakdown

Overview

SnapStack is a single self-contained HTML file built exclusively for mobile viewports (max-width: 480px, no desktop breakpoints). The architecture covers four areas: camera capture, vision-model extraction, manual expense CRUD, and AI insight generation.

Section 1: Mobile-Only Layout Constraints

```
<meta name=viewport content=width=device-width,  
initial-scale=1.0, maximum-scale=1.0, user-scalable=no>  
body { max-width: 480px; margin: 0 auto; }
```

user-scalable=no and fixed initial-scale prevent pinch-zoom from breaking the fixed bottom capture bar. The body max-width centers the app on tablets/desktop browsers without ever attempting a true responsive breakpoint — this app is intentionally not desktop-optimized.

Section 2: Camera Capture Flow

Uses native HTML5 file input with the capture attribute — no custom camera UI or permissions API needed:

```
<input type=file id=cameraInput accept=image/*  
capture=environment onchange=onImageSelected(event)>
```

capture="environment" hints the browser to open the rear-facing camera directly on supporting mobile browsers, falling back gracefully to a file picker on desktop or unsupported browsers.

Section 3: Vision Model Integration

The key technical difference from Days 1-3: this is the first app in the series using a multimodal (vision) local model rather than text-only. The image is base64-encoded and sent in the images array per Ollama's multimodal API contract:

```
async function callOllamaVision(prompt, base64Image) {  
  var r = await fetch(url + '/api/generate', {  
    method: 'POST',  
    body: JSON.stringify({  
      model: model, prompt: prompt,  
      images: [base64Image], stream: false  
    })  
  });  
  return (await r.json()).response;  
}
```

```
}
```

Section 4: Function Reference

Function	Role
onImageSelected(event)	Reads captured file via FileReader, converts to base64, opens modal with preview
extractReceiptData(base64)	Sends image to vision model with JSON-constrained extraction prompt
callOllamaVision(prompt, img)	Multimodal POST to /api/generate with images array
callOllama(prompt)	Text-only POST, used for the spending insight feature
saveExpense()	Validates and persists expense object to localStorage array
runInsight()	Concatenates last 20 expenses, sends pattern-analysis prompt for spending insight
updateSummary()	Computes total spend, count, average, and top category from full expense array

03

HISTORY PAPER

Origin story, design decisions, and iteration log

HISTORY PAPER

SnapStack — Origin, Design Decisions & Build Log

Origin Story

SnapStack is Day 4 of the Mayank Creations Daily Local LLM Startup series, built July 2, 2026. This was also the first day operating under a new standing instruction: all daily series builds are now mobile-only, no desktop layout. SnapStack was chosen specifically because it is a mobile-native use case by definition — photographing a paper receipt is something you do with a phone in your hand, not a desktop browser.

It is also the first app in the series to use a vision-capable local model rather than text-only, expanding the technical range of the series beyond pure text generation into multimodal local AI.

Design Philosophy

Mobile-Only, No Compromise

Rather than building a responsive layout that works "okay" on both desktop and mobile, SnapStack commits fully to a single-column, max-width-480px, thumb-zone-optimized layout. This produces a tighter, more native-feeling experience than a compromised responsive design.

Bottom Sheet Modals

The expense entry modal slides up from the bottom rather than centering on screen — mirroring iOS/Android native modal conventions familiar to mobile users.

Capture-First Interaction

The primary action (scan receipt) is a large, fixed, thumb-reachable button at the screen bottom at all times — never buried in a menu.

Editable AI Output

Vision extraction pre-fills the form but never auto-saves — the user always reviews and can correct merchant/amount/category before committing, since vision-LLM OCR accuracy varies by receipt quality and model size.

Amber/Gold Palette

A warm amber (#ffb347) evokes money/value association, distinct from the previous three days' palettes (purple, orange, teal) — maintaining visual distinction across the growing series.

Build Log — July 2, 2026

Time	Action	Decision
------	--------	----------

T+0:00	New constraint applied	Standing instruction received: all series builds now mobile-only going forward
T+0:10	Idea selection	Chose expense tracking — naturally mobile-first use case, also introduces vision-model capability
T+0:25	Layout design	Single column, max-width 480px, fixed bottom capture bar, bottom-sheet modal
T+0:45	Camera integration	Used native file input with capture=environment instead of custom MediaStream camera UI for simplicity
T+1:05	Vision prompt design	Iterated extraction prompt to reliably return merchant/amount/category as JSON from llava
T+1:30	Category system	8 fixed categories with icon + color mapping for fast visual scanning
T+1:50	Insight feature	Added on-demand spending pattern analysis using last 20 expenses as context
T+2:05	Documentation	Whitepaper, Source Code Paper, History Paper compiled into single PDF

Key Decisions Rejected

Rejected Option	Why Rejected
Responsive desktop layout	Standing instruction now mandates mobile-only builds; desktop breakpoints intentionally omitted
Custom MediaStream camera UI	Native file input capture attribute is simpler, more compatible, and requires no permission-handling
Auto-save on AI extraction	Vision-LLM accuracy varies; user review step prevents bad data from silently entering the ledger
Bank/Plaid integration	Directly contradicts local-only privacy mission
Recurring expense detection	Deferred — adds complexity better suited for a post-MVP iteration with more usage data

What Day 5 Could Build On

- Monthly budget limits per category with overspend alerts
- Multi-receipt batch scanning in one session
- Export to CSV for tax season
- Recurring expense auto-detection (subscriptions)
- Currency conversion for travel expenses
- Voice-entry fallback for when camera/vision model is unavailable

SnapStack — Day 04 of the Daily Local LLM Startup Series
Built by Mayank Swaraj | Mayank Creations | July 2, 2026